COOPERATIVE MULTI-AGENT REINFORCEMENLEARNING FOR INTEGRATED EV–CHARGING STATION–GRID COORDINATION



Bachelor of Technology in Artificial Intelligence &Engineering

Submitted By: **G.DEEPU** **AV.SC.U4AIE23115**

Under the guidance of

Dr. T.K BALAJI

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
AMRITA VISHWAVIDYAPEETAM ,AMARAVATHI ,KURAGALU – 522503

INDIA

TABLE OF CONTENTS

S. No.	Contents	Page No.
	Abstract	2
1	INTRODUCTION AND PROBLEM STATEMENT	3
1.1	Introduction	3
1.2	Problem Statement	4
1.3	Objectives	4
2	BACKGROUND THEORY AND RELATED WORK	5
2.1	Electric Vehicle Charging and State of Charge	5
2.2	Reinforcement Learning	5
2.3	Deep Q-Network (DQN)	6
2.4	Multi-Agent Reinforcement Learning (MARL)	6
2.5	Reward Design for EV Charging Coordination	7
2.6	Related Work and Research Gap	7
3	METHODOLOGY AND IMPLEMENTATION	7
3.1	System Architecture	8
3.2	CityLearn Simulation Environment	8
3.3	EV Agent Configuration and State Representation	9
3.4	Action Space and Safety Masking	9
3.5	DQN Network Architecture	9-10
3.6	Training Algorithm and Workflow	10
3.7	Baseline Charging Strategies	10-11
4	EXPERIMENTAL SETUP AND EVALUATION METRICS	11
4.1	Hardware and Software Requirements	11
4.2	Dataset and Simulation Configuration	11
4.3	Training Parameters and Random Seeds	11
4.4	Evaluation Metrics	11
5	RESULTS AND ANALYSIS	12
5.1	Five-Seed Comparative Evaluation	12
5.2	Grid Import and Peak Load	12
5.3	EV Charging Energy and Service	12
5.4	Effect of Service-Aware Reward Design	12
6	DISCUSSION	13
6.1	Interpretation of Results	13
6.2	Limitations	13
7	CONCLUSION	13
	REFERENCES	14

ABSTRACT

As more people switch to electric vehicles (EVs), charging them without any coordination puts real strain on the power grid, especially when many cars plug in at once. Holding charging back too aggressively causes the opposite problem: a car may not have enough energy when its owner needs to leave. This project looks at how to balance those two pressures.

To do this, I built a cooperative Multi-Agent Reinforcement Learning (MARL) system in the CityLearn simulation environment. Each of eight charging slots is controlled by a Deep Q-Network (DQN) agent. Every agent sees details about its own vehicle, namely the battery state of charge, the energy still required, the time of departure and the charger's limits. It also sees shared signals about overall electricity demand and energy conditions. I tested two cooperative designs: an original version, and a service-aware version that puts extra weight on meeting each vehicle's charging needs.

Both were compared against three baselines: uncontrolled charging, rule-based smart charging and independent DQN agents. Every method was run with five random seeds and judged on grid electricity import, peak load, peak-to-average ratio, energy delivered, final battery level and how many EVs finished charging.

The results show that lower grid demand alone does not mean a strategy works well. The original cooperative model cut grid import and peak load but delivered less energy to the vehicles. The service-aware model charged more, yet it did not reliably improve completion rates or lower peak demand. Overall, the findings suggest that an EV charging system should be designed around grid performance and driver needs together, not one at the expense of the other.

Keywords: Electric Vehicles, Multi-Agent Reinforcement Learning, Deep Q-Network, EV Charging Coordination, CityLearn, Smart Charging, State of Charge, Peak Load Management.

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

Electric vehicles (EVs) are becoming more common, and every new one adds to the electricity load that power grids have to handle. If cars are charged with no coordination, demand can spike at certain hours. If charging is held back too much, though, a driver may come back to a car that isn't ready. A good charging strategy has to protect the grid without letting down the people who rely on their vehicles.

Reinforcement Learning (RL) lets an agent work out what to do by trying actions in an environment and learning from the outcomes. Multi-Agent Reinforcement Learning (MARL) takes this further by having several agents make decisions together so they can pursue a common goal.

In this project, I use Deep Q-Networks (DQN) inside the CityLearn simulation environment to compare five charging strategies: uncontrolled charging, rule-based smart charging, independent DQN, cooperative MARL-DQN and a service-aware version of the cooperative MARL-DQN. Each is assessed on grid electricity import, peak demand, battery state of charge and whether charging is completed. The main question is how far grid demand can be reduced before EV charging needs start to suffer.

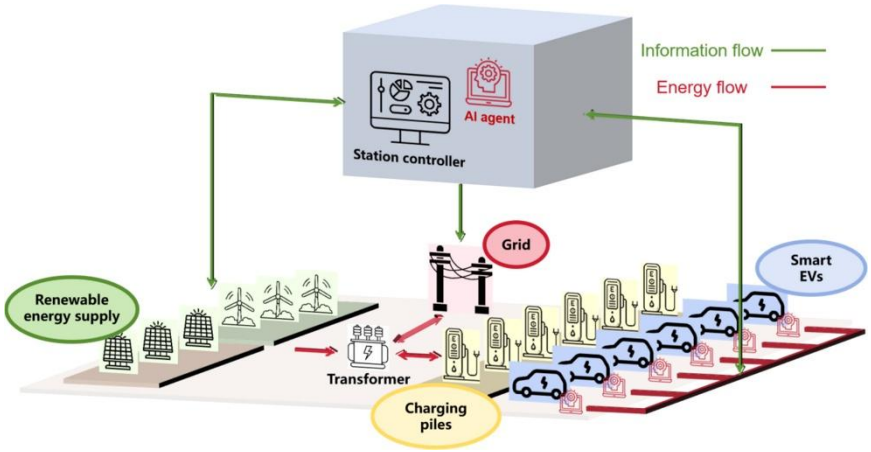


Image Address https://ars.els-cdn.com/content/image/1-s2.0-S2666792426000132-gr3.jpg

1.2 Problem Statement

As electric vehicles (EVs) become more common, it gets harder to match charging demand to what the grid can supply. When many cars charge at the same moment, total demand can jump and push up the peak load. Cutting charging back is not a simple fix either. If the system ignores how much charge each battery needs and when each car is due to leave, some vehicles will not get the energy they need.

Uncontrolled charging makes no attempt to line up vehicle demand with grid conditions. Fixed rule-based strategies are also limited, because they struggle to adjust when charging needs and energy conditions change. Reinforcement Learning offers another route: a charging policy can be trained by interacting with a simulated environment and learning from the results.

This project develops and tests a cooperative multi-agent charging approach built on DQN. Each agent takes into account the vehicle's battery condition, the charge it still requires and its departure time, along with shared information about the electricity system. I compare the approach against conventional charging methods and independently trained agents to see whether it strikes a better balance between managing grid demand and serving the EVs' charging needs.

1.3 Objectives

This project aims to build and test a Multi-Agent Reinforcement Learning (MARL) approach that coordinates EV charging while keeping both the grid and the vehicles' charging needs in view.

To get there, the work is broken into the following objectives:

1. Set up the CityLearn simulation environment with several buildings and EV charging facilities.
2. Describe each of the eight EV charging slots with a fixed-size state vector that combines vehicle-specific details and shared system information.
3. Implement a Deep Q-Network (DQN) that learns charging decisions by interacting with the environment.
4. Build a cooperative MARL-DQN in which all EV agents rely on one shared decision network.
5. Explore a service-aware reward design that places extra weight on battery-charge progress and on meeting each EV's charging requirements.
6. Compare five strategies: uncontrolled charging, rule-based smart charging, independent DQN, cooperative MARL-DQN and service-aware cooperative MARL-DQN.
7. Measure performance using grid import, peak grid load, peak-to-average ratio, charging energy, final battery state of charge and EV completion rate.
8. Run the experiments over five random seeds to check how consistent the results are, and to identify the trade-offs between lowering grid demand and satisfying EV charging needs.

2. BACKGROUND THEORY AND RELATED WORK

2.1 Electric Vehicle Charging and State of Charge

As more people drive electric vehicles (EVs), charging has to be managed with care. When many cars charge at once, grid demand and peak loads climb. Holding charging back has its own cost, since a vehicle may be left without enough energy when its owner needs it. A sound strategy therefore has to weigh what the grid can handle against what each vehicle needs.

State of Charge (SOC) tells us how much energy is left in a battery, expressed as a percentage of its usable capacity. Its value shifts with the charging or discharging power, the length of each action, the battery's capacity and the charging efficiency. Uncontrolled charging can produce sharp demand peaks. Smart charging avoids this by adjusting how and when a vehicle charges based on its needs and the state of the grid.

In this project, every EV agent picks one of five discrete actions: strong charging (+1.0), moderate charging (+0.5), idle (0), moderate discharging (−0.5) or strong discharging (−1.0). A vehicle that is not plugged in can only choose idle, which rules out invalid charging or discharging decisions. The goal is to coordinate charging across several EVs while keeping both grid performance and vehicle service in mind. Lower grid demand means little if vehicles end up without their required charge, so the two are evaluated side by side.

2.2 Reinforcement Learning

Reinforcement Learning (RL) is a branch of machine learning in which an agent learns to make decisions by interacting with an environment, rather than learning only from labelled examples. At each step, the agent looks at the current situation, picks an action, receives a reward and then sees the new state that results. After many rounds of this, it works toward a policy that earns as much cumulative reward as possible over time.

An RL problem is usually described with these elements:

- **Agent:** the decision-maker that chooses actions.
- **Environment:** the system the agent interacts with.
- **State:** the information that describes the current situation.
- **Action:** a choice the agent can make.
- **Reward:** a numerical signal showing how good or bad an outcome was.
- **Policy:** the rule the agent follows to choose actions from the states it observes.

This interaction can be seen as a chain of state transitions. At time step t, the agent observes a state s_t, chooses an action a_t, receives a reward r_t and moves to the next state $s_{(t+1)}$. The aim is to find a policy that gives a high cumulative reward across the whole episode.

The discounted return is commonly written as

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

Here, G_t is the return from time step t, $r_{(t+k+1)}$ is a future reward, and γ is the discount factor, which lies between zero and one. When γ is large, the agent gives more weight to rewards that arrive later. When it is small, the agent focuses mostly on immediate results.

RL suits EV charging coordination because a charging decision affects more than the current time step. Charging a car right away may cover part of its energy need, but it can also add load when the grid is already busy. Waiting may ease the pressure on the grid, yet it can leave too little time to fully charge the vehicle. Through repeated interaction with a simulated environment, an RL agent can learn how to weigh these competing effects.

In this project, reinforcement learning is the overall framework for making decisions. A Deep Q-Network estimates the value of each available charging action, and the cooperative setup pushes agents to take shared system conditions into account when they choose.

2.3 Deep Q-Network (DQN)

Q-learning is value-based reinforcement-learning method this estimates expected return associated with taking an action in a particular state and subsequently following a policy. The action-value function, denoted by $(Q(s,a))$, represents the expected discounted return obtained by selecting action (a) in state (s).

The Q-learning update is defined as

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha\left[r_t + \gamma \max_{a'} Q(s_{t+1},a') - Q(s_t,a_t)\right]$$

Here, α is the learning rate, γ (between zero and one) is the discount factor, and the max term estimates the value of the best action available in the next state.

Tabular Q-learning stops being practical once the state space has many continuous or high-dimensional features. A Deep Q-Network (DQN) gets around this by using a neural network to approximate the action-value function. The network takes a state as input and returns an estimated Q-value for each available action. The agent picks actions based on these estimates, and during training some exploration is added so it can find better decisions it might otherwise miss.

Two mechanisms are commonly used to keep DQN training stable: experience replay and a target network. Experience replay saves past transitions in a replay buffer. At each training step, a batch is sampled from that buffer, so learning does not depend on the order in which experiences happened to occur. The target network supplies a steadier estimate of future Q-values and is refreshed periodically or through another controlled update.

A typical DQN learning target is

$$y_t = r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1},a')$$

where *Q_target* is the target network. For non-terminal transitions, the online network is trained to shrink the gap between its own Q-value estimate and this target. For terminal transitions, the future-value term is simply left out, so the target is just the reward.

In the proposed system, the DQN takes a numerical description of an EV's local operating conditions together with shared system information. Each state has 22 features: 10 local and 12 shared. The network outputs Q-values for the five discrete charging actions, so the chosen action reflects both the individual vehicle's condition and the wider charging and grid context.

All EV agents use one shared DQN model, which is a form of parameter sharing. Instead of keeping a separate network for every EV, the agents learn from a common set of parameters. This reduces model complexity and encourages consistent decisions, though how well it works depends on the state representation, the reward design and the training process.

2.4 Multi-Agent Reinforcement Learning (MARL)

Multi-Agent Reinforcement Learning (MARL) applies reinforcement learning to settings where several agents make decisions that affect one shared system. EV charging fits this approach well. When many vehicles charge at the same moment, grid demand rises, but if their decisions are coordinated, charging can be spread out more evenly. Under independent learning, each agent mostly learns from its own experience. Cooperative learning instead pushes agents toward decisions that serve a common goal. This can be done through shared rewards, shared system information and parameter sharing. This project implements a cooperative DQN with parameter sharing for eight EV charging slots spread across seven CityLearn buildings. Every agent combines local EV information with shared system features, and one common DQN network supports coordinated decisions. The implementation works at the level of individual EVs, so it does not use separate learning agents for EVs, charging stations and the grid.To see how

coordination and reward design shape grid performance and EV charging outcomes, I compare the cooperative model with independent DQN and with a service-aware cooperative variant.

2.5 Reward Design for EV Charging Coordination

Reward design tells an agent what to care about. For EV charging coordination, the reward has to weigh grid demand against what the vehicles need. If it pushes too hard on cutting grid load, EVs may end up undercharged. If it rewards charging completion alone, peak demand may rise.

A cooperative reward can bring together grid performance, EV charging service and any other operating goals. In general form it can be written as:

$$r_t = w_g r_{g,t} + w_s r_{s,t} + w_e r_{e,t}$$

where *r_(g,t)* is the grid-related reward, *r_(s,t)* is the EV-service reward, and *r_(e,t)* is a further operating component. The weights *w_g*, *w_s* and *w_e* set how much each term influences learning. The terms and weights shown here should be matched to the reward function you actually implemented.

This project compares the cooperative MARL-DQN with a service-aware cooperative variant to see how different reward priorities change charging behaviour. Results are judged on grid import, peak demand, peak-to-average ratio, charging energy, final SOC and charging completion. Looking at several metrics together reveals trade-offs that cumulative reward alone would hide.

2.6 Related Work and Research Gap

Researchers have approached EV charging management in several ways, including rule-based methods, optimization techniques, reinforcement learning and multi-agent approaches. Rule-based methods are easy to put in place, but they often cope poorly when grid conditions and vehicle needs change. Optimization methods can handle operational constraints, though how well they work depends on the system model, the information available and the computing cost.

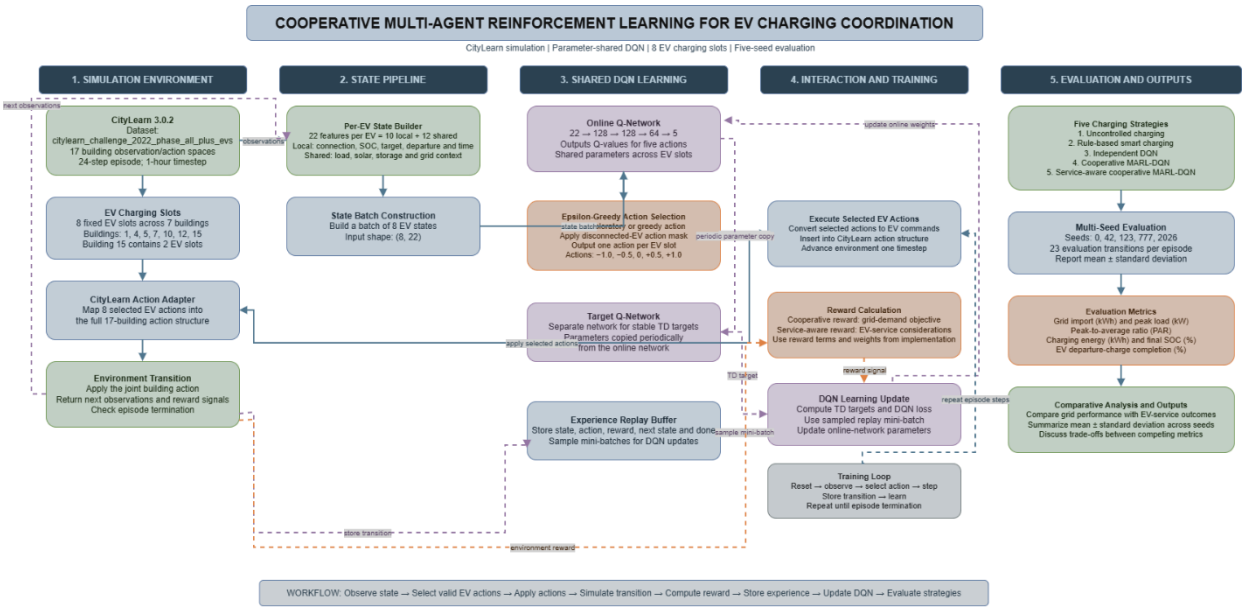
Reinforcement Learning (RL) takes a different route by learning charging decisions from interaction with an environment. Deep Q-Networks (DQN) fit problems with discrete actions, and Multi-Agent Reinforcement Learning (MARL) covers cases where several decision-makers affect the same electricity network. Parameter sharing can also help multiple EV agents learn in a consistent way.The central difficulty is balancing grid efficiency against EV charging service. Lower electricity import or peak demand does not guarantee that vehicles get enough charge. The reverse also holds: high charging completion can push grid demand up. Charging strategies therefore need to be judged on both grid-related and vehicle-related measures.This project tackles that trade-off by comparing five strategies in CityLearn: uncontrolled charging, rule-based smart charging, independent DQN, cooperative MARL-DQN and service-aware cooperative MARL-DQN. Each one is evaluated across several random seeds on grid import, peak demand, peak-to-average ratio, charging energy, final SOC and charging completion.

The main aim is to see how cooperation and reward design shape the balance between grid performance and EV charging needs. The study does not claim that one method is best in every case. Instead, it compares the implemented methods under the same simulation conditions and points out where each is strong, where it falls short and how it could be improved.

3. METHODOLOGY AND IMPLEMENTATION

This chapter describes how the proposed EV charging coordination system is built and evaluated. CityLearn is used to simulate EV charging and see how it affects grid demand. A shared-parameter Deep Q-Network (DQN) lets multiple EV agents learn charging decisions, and two reward designs, cooperative and service-aware, are tested against baseline charging strategies. Every method is run under the same experimental conditions so that grid performance and EV charging outcomes can be compared fairly.

3.1 System Architecture



.

The system decides how to charge eight EV slots, using CityLearn as the simulator and one shared DQN as the decision-maker. At every time step, the EV-specific states and the shared energy-system features are gathered and assembled into a batch of state vectors. The DQN estimates the value of each action, and an action is chosen for every EV, with availability constraints applied.These actions are passed to CityLearn through its action interface. The environment then moves forward and returns new observations and reward signals. Each experience is saved in a replay buffer, and batches sampled from it are used to update the online network, while a target network keeps learning stable. Because the parameters are shared, the architecture has 28,037 trainable parameters, and experience from any EV slot helps train one common policy.The standard cooperative MARL-DQN and the service-aware cooperative MARL-DQN are evaluated alongside independent DQN, uncontrolled charging and rule-based smart charging. All five are compared on grid-demand and EV-service metrics, which shows how coordination and reward design change the results.

3.2 CityLearn Simulation Environment

CityLearn simulates the energy system and lets every EV charging strategy be tested under the same conditions. Because everything runs in simulation, the controller can be evaluated without physical EVs or charging infrastructure.The project uses CityLearn version 3.0.2 with the citylearn_challenge_2022_phase_all_plus_evs dataset. Each episode lasts 24 steps, with a flat interface and rendering turned off. One time step equals one hour, so charging behaviour and electricity demand can be followed across a full day.The environment exposes 17 building-level observation spaces and 17 action spaces. This study concentrates on eight EV slots located in seven buildings: Building_1, Building_4, Building_5, Building_7, Building_10, Building_12 and Building_15. Each building has a single EV slot, apart from Building_15, which has two.At every time step, the controller gathers the relevant observations and builds the state input for the DQN. The chosen charging actions are then applied in the environment, which updates its state and returns new observations and reward signals. This loop repeats until the episode ends.Five strategies are evaluated in the same simulation setup: uncontrolled charging, rule-based smart charging, independent DQN, cooperative MARL-DQN and service-aware cooperative MARL-DQN. They are compared on electricity import, peak demand, charging energy, final battery state of charge (SOC) and EV charging completion.

3.3 EV Agent Configuration and State Representation

The system treats each of the eight EV slots, spread across seven buildings, as its own decision-making agent. Even though every agent stands for a different vehicle, all of them use the same DQN network. This parameter sharing lets the agents learn one common policy while still reacting to their own operating conditions. Each EV state has 22 features: 10 local features that describe the vehicle and 12 shared features that capture wider energy-system conditions. They are joined into one state vector and fed to the DQN, so the agent can account for both its own charging needs and the system context around it. The state is written as

$$s_t = \left[s_t^{\text{local}}, s_t^{\text{shared}}\right]$$

where *s_t* is the complete state at time step *t*, *s_t^local* holds the local EV features, and *s_t^shared* holds the shared system features. Together they give a state dimension of 22.The shared DQN uses this state to estimate the value of each available charging action. As a result, all eight EV slots draw on one set of network parameters instead of eight separate networks. The exact feature definitions should follow your final state-construction code.

3.4 Action Space and Safety Masking

The controller uses a discrete action space with five choices, allowing the DQN to estimate a value for each possible charging or discharging action.

Action value	Meaning
−1.0	Strong discharging
−0.5	Moderate discharging
0	Idle operation
+0.5	Moderate charging
+1.0	Strong charging

Positive values represent charging, negative values represent discharging, and zero represents idle operation. These values are discrete action labels, not a direct power measurements in kW. At each time step, the DQN selects an action using its estimated action values and the implemented exploration policy. **Safety masking** restricts disconnected EVs to the zero action, preventing charging or discharging commands when a vehicle is unavailable. This ensures that action selection reflects the EV's current connection status and the constraints of the simulation environment.

3.5 DQN Network Architecture

A Deep Q-Network (DQN) estimates the expected long-term return of each available charging action using the action-value function (Q(s,a)), where (s) is the current state and (a) is a candidate action.

The network receives a 22-dimensional EV state vector and produces five output values, one for each discrete action. The action-selection rule is expressed as:

$$a_t = \arg\max_{a \in A} Q(s_t, a)$$

where (a_t) is the selected action, (A) is the available action set, and (Q(s_t,a)) is the estimated action value. During training, the exploration strategy may select a different action to encourage learning.

DQN uses **experience replay** to learn from stored transitions containing the state, action, reward, next state, and termination information. A separate target network provides estimates for calculating learning targets, helping stabilise updates to the online network.

The project uses parameter sharing across eight EV slots, allowing the same network to learn from different EV experiences. The implemented network contains **28,037 trainable parameters**, providing a common action-value estimation mechanism for EV-level charging coordination.

3.6 Training Algorithm and Workflow

The DQN learns by interacting with the CityLearn environment over and over. At each time step, the system gathers the EV observations, builds the 22-feature state, chooses a charging action for each EV, and records the reward and the next state. These transitions go into replay memory and are used to update the shared DQN, while a target network keeps the learning steady.

The training workflow runs as follows:

1. Set up CityLearn, the EV components and the discrete action space.
2. Create the shared DQN, the target network and the replay memory.
3. Reset the environment and build the initial state for each EV.
4. Choose actions with the implemented exploration strategy, keeping disconnected EVs on idle.
5. Apply the actions, then collect the rewards, next states and termination information.
6. Save the transitions in replay memory and update the DQN once the training conditions are met.
7. Refresh the target network on the configured schedule.
8. Repeat until the episode ends, then carry on through the configured training runs.
9. Evaluate the selected methods with five random seeds: 0, 42, 123, 777 and 2026.

Exploration lets the agent try different actions and learn from what happens. As experience builds up, the DQN's action-value estimates improve. Evaluation uses 23 hourly transitions per episode, and results are summarized across the five seeds as a mean and standard deviation.

3.7 Baseline Charging Strategies

Five charging strategies are tested so that conventional control, independent learning, cooperative learning and service-aware decision-making can be compared directly.

1.UncontrolledCharging

This strategy relies on the environment's built-in uncontrolled charging, with no coordinated learning policy. It acts as the reference point for judging grid demand and EV charging outcomes.

2.Rule-BasedSmartCharging

Here, charging follows a fixed set of predefined rules and nothing is learned from experience. It serves as a benchmark for how much reinforcement learning adds over fixed decision rules.

3.IndependentDQN

Independent DQN uses value-based reinforcement learning but without the shared network used in the cooperative method. Comparing it with the cooperative approach shows what cooperative learning contributes to EV charging decisions.

4.CooperativeMARL-DQN

This method gives the EV agents one shared DQN network and combines local EV information with shared system features. As a result, each charging decision takes wider system conditions into account.

5.Service-AwareCooperativeMARL-DQN

This method builds on the cooperative approach by changing the reward design to put more weight on EV charging service. It is used to explore the trade-off between grid performance and charging outcomes.

Every strategy is assessed on grid import, peak load, peak-to-average ratio (PAR), charging energy, final state of charge (SOC) and charging completion. Results are reported as the mean and standard deviation across five random seeds.Taken together, these baselines show where each approach is strong and where it falls short, especially in the trade-off between cutting grid demand and meeting EV charging needs.

4. EXPERIMENTAL SETUP AND EVALUATION METRICS

This chapter presents the simulation configuration, computational environment, training setup, and evaluation metrics used to compare the five EV charging strategies.

4.1 Hardware and Software Requirements

The project was implemented using Python and PyTorch, with CityLearn providing the simulation environment. Training was performed using an NVIDIA Tesla T4 GPU with CUDA support.

Component	Configuration
Programming language	Python 3.13.15
Deep-learning framework	PyTorch 2.11.0+cu130
GPU	NVIDIA Tesla T4
Simulation framework	CityLearn 3.0.2
Dataset	citylearn_challenge_2022_phase_all_plus_evs

4.2 Dataset and Simulation Configuration

The experiments use the CityLearn dataset citylearn_challenge_2022_phase_all_plus_evs. The environment is configured for 24 time steps with a one-hour interval, while evaluation uses 23 transitions per episode.

The system includes eight EV slots across seven buildings. Each EV is represented by a 22-dimensional state vector containing 10 local features and 12 shared system features. The controller selects from five discrete charging, discharging, and idle actions. Disconnected EVs are restricted to the idle action.

4.3 Training Parameters and Random Seeds

The cooperative DQN uses a shared neural network containing 28,037 parameters to estimate the values of the five available actions. The experiments compare uncontrolled charging, rule-based smart charging, independent DQN, cooperative MARL-DQN, and service-aware cooperative MARL-DQN.

To account for variability between runs, the experiments use five random seeds: 0, 42, 123, 777, and 2026. Performance is summarized using the mean and standard deviation across these runs. The exact DQN hyperparameters should be included only after verifying them against the final notebook configuration.

4.4 Evaluation Metrics

The strategies are evaluated using grid-related and EV-service metrics to assess their overall performance.

- **Grid import (kWh):** Total electricity drawn from the grid during evaluation.
- **Peak grid demand (kW):** Maximum grid power observed during the evaluation period.
- **Peak-to-average ratio (PAR):** Ratio of maximum grid demand to average grid demand.

$$\mathrm{PAR} = \frac{P_{\max}}{P_{\mathrm{avg}}}$$

- **Charging energy (kWh):** Total energy delivered to EVs.

- **Final state of charge (SOC, %):** Battery charge remaining at the end of evaluation.
- **Charging completion (%):** Percentage of eligible charging requirements satisfied according to the implemented completion criterion.
- **Mean reward:** Average reward obtained under the method's reward function.

Results are summarized across the five random seeds using the mean and standard deviation. The mean is calculated as

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

where (x_i) is the metric value for run (i), and (n=5).

Considering these metrics together is essential because reducing grid demand does not necessarily mean that EV charging requirements are satisfied. The combined evaluation therefore measures both electricity-system performance and charging service quality.

5. RESULTS AND ANALYSIS

5.1 Five-Seed Comparative Evaluation

The five charging strategies were evaluated using five random seeds. Results are reported as mean ± standard deviation across grid-performance and EV-service metrics.

Table 5.1. Comparative performance of charging strategies

Method	Grid Import (kWh)	Peak Load (kW)	PAR	Charging Energy (kWh)	Final SOC (%)	Completion (%)
Uncontrolled Charging	836.96 ± 51.53	101.44 ± 5.59	2.790 ± 0.107	1347.60 ± 335.07	**90.82 ± 8.48**	**82.50 ± 16.77**
Rule-Based Smart Charging	576.01 ± 21.95	60.10 ± 5.41	**2.408 ± 0.297**	326.26 ± 16.92	62.54 ± 8.22	10.36 ± 10.59
Independent DQN	748.19 ± 120.81	96.30 ± 4.62	3.023 ± 0.502	986.82 ± 208.23	87.51 ± 8.44	75.00 ± 19.76
Cooperative MARL-DQN	**301.65 ± 55.25**	**48.68 ± 12.51**	3.690 ± 0.599	228.68 ± 34.80	83.61 ± 4.22	63.33 ± 9.03
Service-Aware Cooperative MARL-DQN	791.66 ± 39.13	99.02 ± 2.92	2.881 ± 0.125	926.80 ± 32.92	85.80 ± 0.88	60.00 ± 5.59

5.2 Grid Import and Peak Load

Cooperative MARL-DQN achieved the lowest reported grid import (301.65 kWh) and peak load (48.68 kW). However, it also delivered the least charging energy (228.68 kWh). Therefore, reduced grid demand alone does not establish better overall performance.

5.3 EV Charging Energy and Service

Uncontrolled charging achieved the highest reported final SOC (90.82%) and completion rate (82.50%), but it also produced the highest peak load. The cooperative method reduced grid demand at the cost of lower charging energy, while the service-aware method increased charging energy without improving completion over the original cooperative method.

5.4 Effect of Service-Aware Reward Design

Compared with Cooperative MARL-DQN, the service-aware variant increased charging energy from 228.68 to 926.80 kWh and grid import from 301.65 to 791.66 kWh. Peak load also increased from 48.68 to 99.02 kW, while completion decreased from 63.33% to 60.00%. These results indicate that the revised reward changed charging behaviour but did not improve every service metric.

6. DISCUSSION

6.1 Interpretation of Results

The experiments show that Cooperative MARL-DQN recorded the lowest grid import (301.65 ± 55.25 kWh) and the lowest peak load (48.68 ± 12.51 kW) of the five charging strategies. Relative to uncontrolled charging, that is a drop of roughly 64.0% in grid import and 52.0% in peak load.

This came at a cost, though. The cooperative method delivered less charging energy than uncontrolled charging and also completed fewer EV charging sessions. The service-aware variant raised charging energy, but it also pushed grid import and peak load back up, and it did not improve completion over the original cooperative method. These results illustrate the trade-off between cutting grid demand and meeting EV charging needs.

In short, cooperative reinforcement learning can lower grid demand under the simulation conditions tested here, but the reward design needs further tuning before it balances grid performance with EV service quality.

6.2 Limitations

This study has several limitations that should be kept in mind when interpreting the results.

- **Simulation only:** All results come from CityLearn. They have not been checked against real charging-station data, so real-world performance may differ.
- **Short horizon:** Experiments used a 24-step setup with 23 evaluated transitions, which limits what can be said about long-term behaviour.
- **Reward sensitivity:** The service-aware reward led to more charging activity, but it did not reliably improve EV completion.
- **Few evaluation runs:** Five random seeds give a first impression of variability, but they are not enough to claim broad statistical superiority.
- **Narrow control scope:** The cooperative controller mainly makes EV-level charging decisions. Separate charging-station and grid agents were not trained independently in the evaluated implementation.

7. CONCLUSION

This project built and tested DQN-based EV charging strategies in the CityLearn simulation environment. Five methods were compared over five random seeds, using grid import, peak load, PAR, charging energy, final SOC and EV completion rate.Cooperative MARL-DQN recorded the lowest grid import and peak load, cutting them by about 64.0% and 52.0% compared with uncontrolled charging. These gains came with less charging energy and lower EV completion, which shows that reducing grid demand alone does not prove a charging strategy is effective. The study lays a simulation-based groundwork for cooperative EV charging control and underlines the need to balance grid efficiency with what vehicles require. Future work should refine the reward design, add stronger requirements for charge at departure, and test the controller over longer horizons and in more scenarios.

REFERENCES

[1] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015. https://doi.org/10.1038/nature14236

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. http://incompleteideas.net/book/the-book-2nd.html

[3] J. R. Vázquez-Canteli, S. Dey, G. Henze, and Z. Nagy, “CityLearn: Standardizing research in multi-agent reinforcement learning for demand response and urban energy management,” 2020. https://doi.org/10.48550/arXiv.2012.10504

[4] J. R. Vázquez-Canteli, G. Henze, and Z. Nagy, “MARLISA: Multi-agent reinforcement learning with iterative sequential action selection for load shaping of grid-interactive connected buildings,” in *Proceedings of BuildSys*, pp. 170–179, 2020. https://doi.org/10.1145/3408308.3427604

[5] Z. Wan, H. Li, H. He, and D. Prokhorov, “Model-free real-time EV charging scheduling based on deep reinforcement learning,” *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 5246–5257, 2019. https://doi.org/10.1109/TSG.2018.2879572

[6] F. L. da Silva, C. E. H. Nishida, D. M. Roijers, and A. H. Reali Costa, “Coordination of electric vehicle charging through multiagent reinforcement learning,” *IEEE Transactions on Smart Grid*, vol. 11, no. 3, pp. 2347–2356, 2020. https://doi.org/10.1109/TSG.2019.2952331

[7] “Development and evaluation of a smart charging strategy for an electric vehicle fleet based on reinforcement learning,” *Applied Energy*, vol. 285, article 116382, 2021. https://doi.org/10.1016/j.apenergy.2020.116382

[8] “Multi-agent deep reinforcement learning approach for EV charging scheduling in a smart grid,” *Applied Energy*, vol. 328, article 120111, 2022. https://doi.org/10.1016/j.apenergy.2022.120111

[9] Z. Zhao, C. K. M. Lee, X. Yan, and H. Wang, “Reinforcement learning for electric vehicle charging scheduling: A systematic review,” *Transportation Research Part E: Logistics and Transportation Review*, vol. 190, article 103698, 2024. https://doi.org/10.1016/j.tre.2024.103698

[10] H. M. Abdullah, A. Gastli, and L. Ben-Brahim, “Reinforcement learning based EV charging management systems—A review,” *IEEE Access*, 2021. https://doi.org/10.1109/ACCESS.2021.3064354